COMMAND SEQUENCE, SELECTION AND ITERATION

In Unit 1 you learnt that the three key building blocks of algorithms are command sequence, selection and iteration. In this unit, you will have the opportunity to implement these constructs in the high-level programming language you are studying.

SKILLS CRITICAL THINKING, PROBLEM SOLVING, CREATIVITY

ACTIVITY 3

UNDERSTANDING ALGORITHMS

Read the following algorithm written in pseudocode and then answer the questions below.

```
RECEIVE number1 FROM (INTEGER) KEYBOARD
RECEIVE number2 FROM (INTEGER) KEYBOARD
SET result1 TO number1 / number2
SEND result1 TO DISPLAY
SET result2 TO number1 MOD number2
SEND result2 TO DISPLAY
SET result3 TO number1 DIV number2
SEND result3 TO DISPLAY
```

1 What does this algorithm do?

2 What is the output of the algorithm, given the following inputs:
   a 4, 2
   b 10, 3
   c 20, 6?

Implement this algorithm in the high-level language you are studying.

SELECTION

The selection construct is used to create a branch in a program. The computer selects which branch to follow based on the outcome of a condition, using an `IF...THEN...ELSE` statement. For example, in pseudocode:

```
IF day = 'Saturday' OR day = 'Sunday' THEN
  SET alarm TO 11
ELSE
  SET alarm TO 8
END IF
```

SUBJECT VOCABULARY

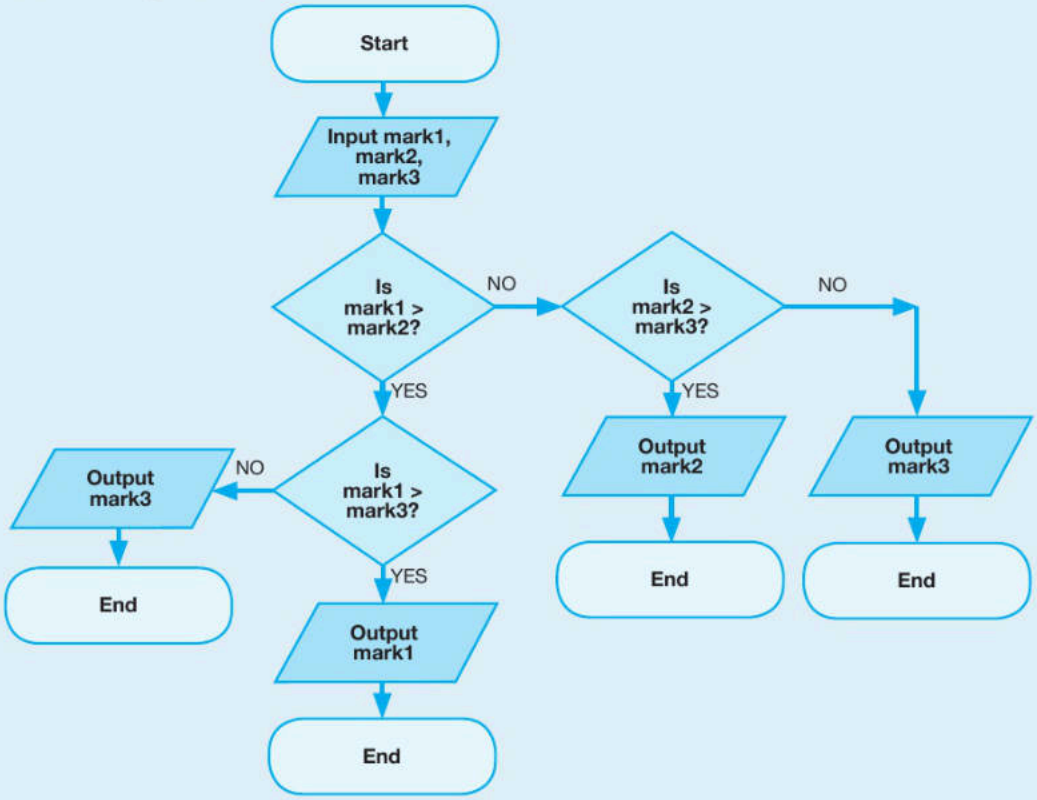
nested IF statement a nested IF statement consists of one or more IF statements placed inside each other. A nested IF is used where there are more than two possible courses of action

A standard `IF...THEN...ELSE` statement provides two alternatives. If there are more than two, then in Pearson Edexcel pseudocode a nested `IF` must be used. However, many high-level programming languages have an additional built-in selection construct that does away with the need for a **nested IF statement**.

WORKED EXAMPLE

A learner handed in three homework assignments, which were each given a mark out of 10. All the marks were different. Write an algorithm that would print out the highest mark.

Figure 2.1 shows the algorithm expressed as a flowchart.



▲ **Figure 2.1** Flowchart of an algorithm to print out the highest homework mark

HINT

When you are creating nested `IF` statements, you have to ensure that each one is completed with an `END IF` statement at the correct indentation level. Some programming languages do not need an `END IF` statement and just use the indentation levels to indicate when statements are grouped.

In Pearson Edexcel pseudocode, this algorithm could be expressed as:

```
RECEIVE mark1 FROM KEYBOARD
RECEIVE mark2 FROM KEYBOARD
RECEIVE mark3 FROM KEYBOARD
IF mark1 > mark2 AND mark1 > mark3 THEN
    SEND mark1 TO DISPLAY
ELSE
    IF mark2 > mark1 AND mark2 > mark3 THEN
        SEND mark2 TO DISPLAY
    ELSE
        IF mark3 > mark1 AND mark3 > mark2 THEN
            Send mark3 TO DISPLAY
        END IF
    END IF
END IF
```

In Python, Java and C# this does not have to be done as they have an 'else if' statement.

In Python the 'else if' statement is `elif` and the algorithm above could be:

```
mark1 = input('Please enter the first mark')
mark2 = input('Please enter the second mark')
mark3 = input('Please enter the third mark')

if mark1 > mark2 and mark1 > mark3:
    print(mark1)
elif mark2 > mark1 and mark2 > mark3:
    print(mark2)
elif mark3 > mark1 and mark3 > mark2:
    print(mark3)
```

Here is the algorithm in Java:

```
import java.util.*;
class Main {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Please enter the first mark: ");
    int mark1 = scanner.nextInt();
    System.out.print("Please enter the second mark: ");
    int mark2 = scanner.nextInt();
    System.out.print("Please enter the third mark: ");
    int mark3 = scanner.nextInt();
    scanner.close();

    if (mark1 > mark2 && mark1 > mark3) {
      System.out.print(mark1);
    } else if (mark2 > mark1 && mark2 > mark3) {
      System.out.print(mark2);
    } else if (mark3 > mark1 && mark3 > mark2) {
      System.out.print(mark3);
    }
  }
}
```

And in C#:

```
string mark1String, mark2String, mark3String;
int mark1, mark2, mark3;

Console.WriteLine("Please enter the first mark");
mark1String = Console.ReadLine();    // read input into string
mark1 = int.Parse(mark1String);      // convert string to integer

Console.WriteLine("Please enter the second mark");
mark2String = Console.ReadLine();    // read input into string
mark2 = int.Parse(mark2String);      // convert string to integer
```

```
Console.WriteLine("Please enter the third mark");
mark3String = Console.ReadLine();    // read input into string
mark3 = int.Parse(mark3String);      // convert string to integer

if (mark1 > mark2 && mark1 > mark3)
{
    Console.WriteLine(mark1);
}
else if (mark2 > mark1  && mark2 > mark3)
{
   Console.WriteLine(mark2);
}
else if (mark3 > mark1 && mark3 > mark2)
{
   Console.WriteLine(mark3);
}
```

These examples use **relational** and **logical operators**.

SUBJECT VOCABULARY

relational operator an operator that tests the relationship between two entities

logical operator a Boolean operator using AND, OR and NOT

RELATIONAL OPERATORS

The relational operators are used to compare two values and in Python, Java and C# are all the same.

RELATIONAL OPERATOR	PYTHON, JAVA, C#
Equal to	==
Greater than	>
Greater than or equal to	>=
Less than	<
Less than or equal to	<=
Not equal to	!=

ACTIVITY 4

Look at the following algorithm and answer the questions.

```
IF score <= highScore THEN
    SEND 'You haven't beaten your high score.' TO DISPLAY
ELSE
    SEND 'You've exceeded your high score!' TO DISPLAY
END IF
```

What is the output of the algorithm when

- `score` = 5 and `highScore` = 10?
- `score` = 20 and `highScore` = 10?
- `score` = 15 and `highScore` = 15?

LOGICAL OPERATORS

AND

If two conditions are joined by the `AND` operator, then they must both be true for the whole statement to be true.

OR

If two conditions are joined by the `OR` operator, then either one must be true for the whole statement to be true.

NOT

The `NOT` operator reverses the logic of the `AND` and `OR` statements. The statement `IF A = 3 AND B = 6` will be true only if the conditions are met, i.e. A and B are both equal to the values stated.

The statement `IF NOT (A = 3 AND B = 6)` will be true whenever both A and B are NOT equal to the values stated, i.e. either or both are not equal to those values.

Logical operators in high-level languages.

LOGICAL OPERATOR	PYTHON	JAVA	C#
AND	and	&	&&
OR	or	|	||
NOT	not	!	!

ACTIVITY 5

A driving school uses this rule to estimate how many lessons a learner will require.

- Every learner requires at least 20 lessons.
- Learners over the age of 18 require more lessons (two additional lessons for each year over 18).

Create a program in a high-level language that inputs a learner's age and calculates the number of driving lessons they will need.

LOOPS

A loop is another name for an iteration. Loops are used to make a computer repeat a set of instructions more than once. There are two types of loop: definite and indefinite.

A definite loop is used when you know in advance how often the instructions in the body of the loop are to be repeated. For example, if you want the computer to display a character on the screen for a fixed amount of time and then remove it.

An indefinite loop is used when the number of times a loop will need to be repeated is not known in advance. For example, if you want to give a user the option of playing a game as often as they want. Indefinite loops are repeated until a specified condition is reached.

Every programming language has a number of built-in loop constructs. You will need to explore the ones provided in the language you are studying.